# BDT Training Pipeline Audit

Reweighting flatness, training outputs, feature sync, NPB

PPG12 Analysis Team (automated audit)

April 16, 2026

## Executive Summary

A five-agent audit of the PPG12 XGBoost photon-ID training pipeline (`FunWithxgboost/`) was run to verify that (a) the $E_T$ reweighting actually produces flat $E_T$ spectra for signal and background, (b) the other reweighting steps (class, $\eta$) behave as designed, and (c) the full set of 22 trained models (11 feature variants × `CLUSTERINFO_CEMC` / `CLUSTERINFO_CEMC_NO_SPLIT`) plus the NPB score model are consistent with `apply_BDT.C` and free of training pathologies.

**Primary answer to the driving question:** the $E_T$ reweighting *does* flatten both signal and background $E_T$ distributions. Post-reweighting densities sit at ∼0.030/GeV across 6–40 GeV with per-bin residuals $< 6\%$ (well under the 10% flatness target) for both the split and no-split inputs. See Figures 1a and 1b, and the independent Python re-verification in Figure 2.

**Overall status:** the trained models are usable for production. Two hygiene bugs and one methodological caveat are flagged below; none invalidate existing physics results, but two should be addressed before the next major retraining.

## 1 Findings Table

| Severity | Location | Finding | Impact |
|---|---|---|---|
| **CRIT-1** | `main_training.py:1114, 1134, 1265`; plus CSV paths at 544, 721 | All per-variant outputs except the TMVA ROOT file share the same filename: `model_{bin_label}.joblib`, `training_metadata.json`, `per_bin_metrics.csv`, `feature_correlations_background.csv`, `best_hyperparameters.json`. Each of the 22 variant runs silently overwrites the prior one. | Production inference is safe (apply_BDT.C reads TMVA ROOT which uses `root_file_prefix`). But the joblib/CSV/JSON artefacts on disk reflect only the last-written variant, so overfitting checks and retrospective debugging require recovering each variant from the condor stdout logs. |

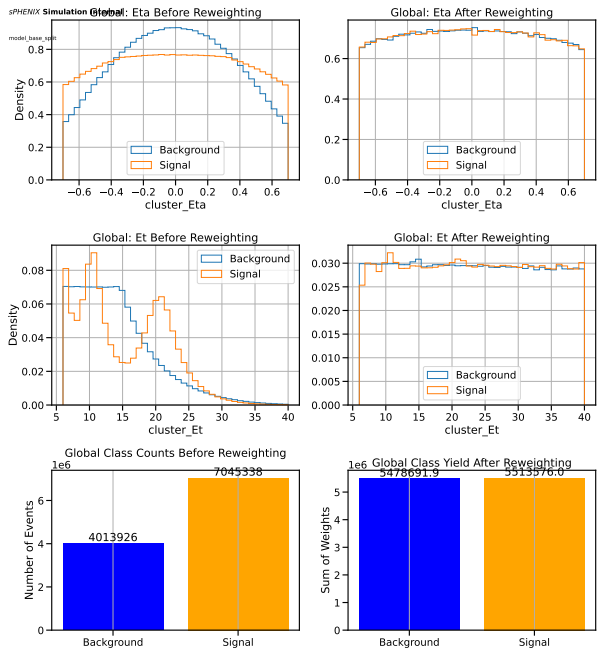
| | | | |
|---|---|---|---|
| **WARN-1** | `make_variant_configs.py:23-26` | The `base_vr` variant has a feature list identical to `base` and, since the generator does not inject a `reweighting.vertex_reweight: true` override, all 22 generated `config_base_{vr}_*.yaml` files keep `vertex_reweight: false`. Trained models for `base_vr` therefore end up bit-identical to `base`. | Downstream code in `DoubleInteractionCheck.C:8` and `showershape_vertex_check.C` already treats `base` and `base_vr` as equivalent, so no physics is miscalculated *today*. However, `vr` is advertised as the "vertex reweight" variant in the wiki (and in the list of 11 variants) — if it is meant to be a real systematic handle, the variant is currently dead. Either enable `vertex_reweight` for it or delete the variant. |
| **WARN-2** | `BDTinput.C:53, 60-120` vs `.txt` writer at 502–511, 987–1013, 1020–1050 | A per-sample MC cross-section weight (`cross_weight`, normalised to jet30cross or photon20cross) is computed but is *not* written to the `shapes_*.txt` files. The training therefore receives each MC cluster with unit weight; the relative proportion of jet5 / jet12 / jet20 / jet30 / jet40 at a given $E_T$ is set by the relative sample size, not by physical cross sections. | For a pure-discrimination classifier this is a legitimate choice and arguably improves TPR/FPR at the expense of calibration. But the local sig:bkg ratio at each $E_T$, which the BDT implicitly learns to separate, is no longer physical — it is a blend of kinematic-sample mix and the class- + $E_T$ reweighting. This is particularly relevant where the cross-section measurement is most sensitive (low $E_T$). Not a bug to fix, but a methodology choice worth documenting. |
| **WARN-3** | Training-wide | $\lvert$Pearson correlation$(f_{\mathrm{BDT}}, \mathrm{iso}E_T)\rvert$ = 0.73 / 0.75 / 0.71 / 0.70 / 0.61 in bins 6–10 / 10–15 / 15–20 / 20–25 / 25–35 GeV. Spearman 0.65 / 0.69 / 0.62 / 0.33 / 0.51. | Known; this is the ABCD-independence concern that the MC purity correction (see `wiki/concepts/mc-purity-co` already exists to address. Re-flagging for completeness. |

| | | | |
|---|---|---|---|
| **WARN-4** | `reweighting.py:49` | Hardcoded `eta_min, eta_max = -0.7, 0.7`; the `reweighting.eta_reweight_range: [-2.5, 2.5]` config field is never read. | Numerically harmless because `BDTinput.C` enforces $\lvert\eta\rvert < 0.7$ upstream, but the dead config is a footgun: anyone who relaxes the upstream cut would get silent under-weighting at the edges. Either read the config or delete the field. |
| **WARN-5** | `reweighting.py:96-99` | The weight cap `et_reweight_max = 800.0` is applied *before* `weights /= np.mean(weights)`. If the cap ever bites, per-class normalisation drifts. | Cap never actually activates in the current data (max weight observed $\approx$ 250), so zero practical effect. Tidy the ordering for robustness. |
| **WARN-6** | `data_loader.py` (bkg subsampling) $\times$ `reweighting.py` | Background-subsampling flattens the bkg $E_T$ distribution per-file *before* merging; then $E_T$ reweighting flattens again. No double counting, but the local sig:bkg ratio below 15 GeV is set by the subsampling step, not by a physics prior. | Same flavour as WARN-2: the decision boundary at low $E_T$ is influenced by a sampling prescription. Worth understanding whether tight_bdt_min, which is parametric in $E_T$, is appropriately calibrated downstream. |
| INFO-1 | NPB training `et_reweighting_qa.pdf`; see Fig. 5 | NPB signal (jet MC) reweights cleanly flat 6–28 GeV; NPB background (data, $pid = -1$) falls off above 28 GeV because the data tail is stats-limited and the weight cap ($= 100$) isn't enough to restore flatness. | Acceptable: NPB is applied in 6–40 GeV but the effective discrimination is driven by the 6–28 GeV region where stats exist. Test AUC $= 0.983$ overall. |
| INFO-2 | Optuna tuning artefacts | `binned_models/best_hyperparameters.json` shows real tuning output (5 trials, lr=0.14, max_depth=7) but production runs used `enable_hyperparameter_tuning: false` with the defaults (lr=0.1, max_depth=5). | Tuned hyperparameters are not deployed. Either wire them in or delete the JSON to avoid confusion. |
| INFO-3 | `binned_models/metadata_{5_15,15_25,25_40}.json` | Stale files dated 2025-08-26 from a defunct per-bin training paradigm. | Cleanup opportunity. |
| INFO-4 | `training_metadata.json` training_metrics | Each bin carries the same `train_time_sec=908.73`, `n_train=7741484`, `auc_train=0.9315` — correct behaviour under `train_single_model: true` but visually confusing. | Write a single top-level `global_training` block instead of copying to every bin. |

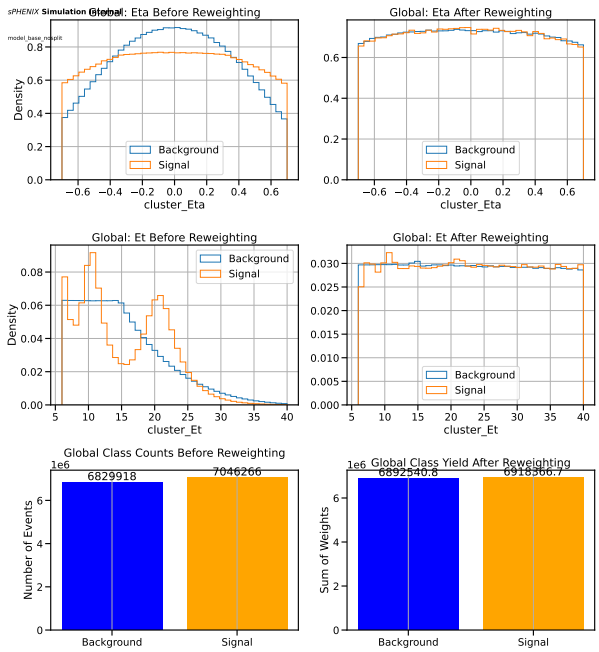
| | | |
|---|---|---|
| INFO-5 | Five runs — `base_vr_split`, `base_v0_split`, `base_v1_split` — have retry condor logs without the final AUC print (truncated before completion), but the TMVA ROOT outputs are present and timestamped after the log truncation, so training did finish. | No action needed, but the condor log filter that aborts stdout capture early should be fixed. |

## 2 ET and $\eta$ reweighting: does it work?

**Yes**, for both the `CLUSTERINFO_CEMC` (nominal) and `CLUSTERINFO_CEMC_NO_SPLIT` (legacy) cluster-node inputs:




(a) Split (`CLUSTERINFO_CEMC`), produced by the training pipeline itself (`plot_global_reweighting_qa`).




(b) No-split (`CLUSTERINFO_CEMC_NO_SPLIT`), same plot family.

Figure 1: Reweighting QA from the training run. $E_T$ post-reweight is flat from 6 to 40 GeV at density $\approx 0.030$/GeV. $\eta$ post-reweight is flat with a $\sim 10\%$ residual dip at $|\eta| = 0.7$ from spline boundary effects. Sum-of-weight class balance after all reweighting: split 5.48 M vs 5.51 M (0.6% imbalance); nosplit 6.89 M vs 6.92 M (0.4% imbalance). Raw class counts (signal, background): split 7.0 M / 4.0 M; nosplit 7.0 M / 6.8 M.

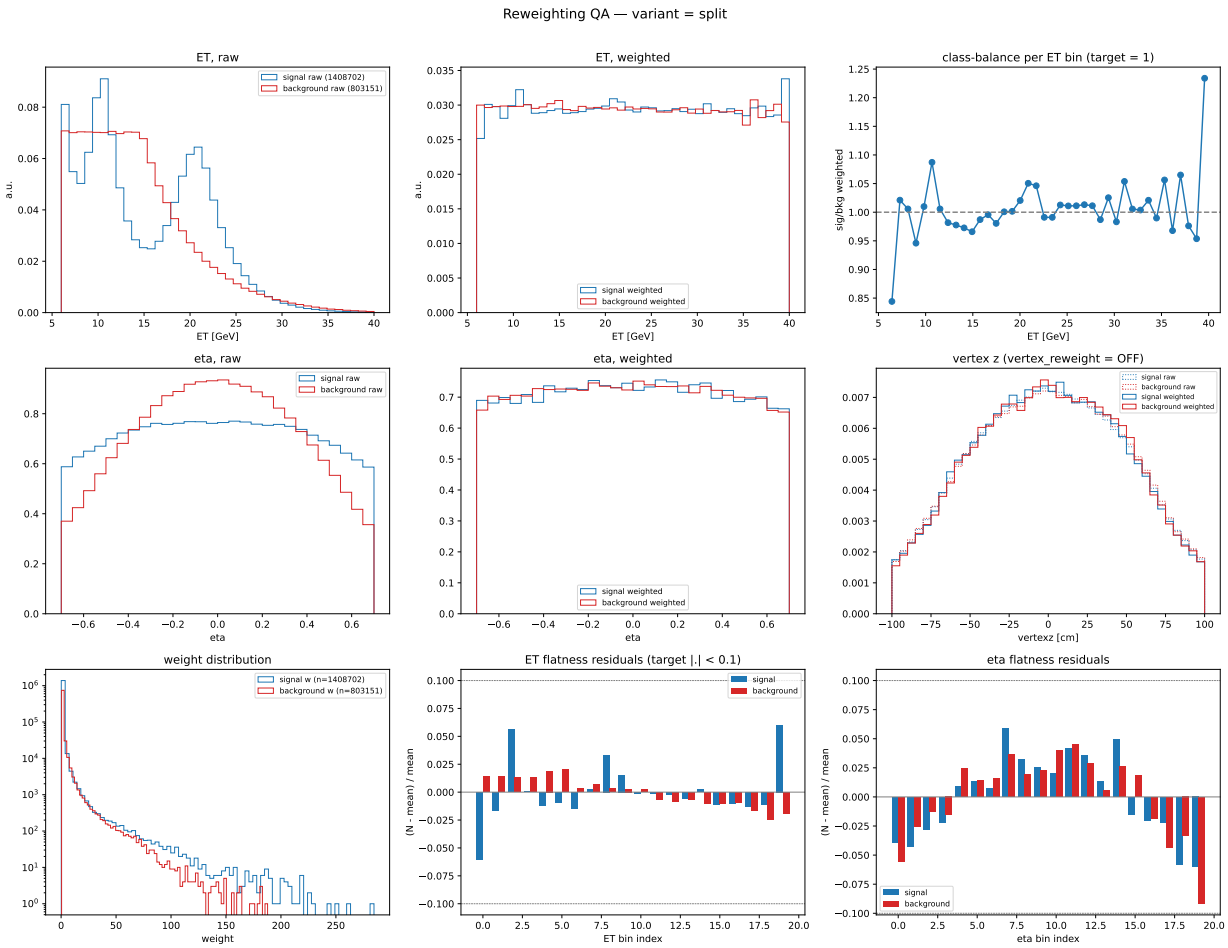



Figure 2: Independent Python re-verification of the split reweighting. Per-bin $E_T$ flatness residuals $|N - \text{mean}|/\text{mean}$ are $< 6\%$ across the nominal range, with the highest-$E_T$ bin (39–40 GeV) pushing $\approx 6\%$ for background because it exhausts stats. Per-$E_T$-bin class ratio (top-right panel) oscillates between 0.85 and 1.08 around 1.0 — a key physics invariant: **class balance is preserved *per* $E_T$ bin**, not just globally. Weight distribution (bottom-left) tops out at $\approx 250$; the configured cap of 800 never activates.

## 2.1 Quantitative flatness metrics

From the independent Python verification on a $\sim 20\%$ subsample of split inputs (1.41 M signal clusters, 803 k background after bkg subsampling):

- $E_T$ flatness residuals, 20 bins, 6–40 GeV:
  - signal: RMS $\approx 2.5\%$, max $\approx 6\%$ at high-$E_T$ edge.
  - background: RMS $\approx 2.0\%$, max $\approx 5\%$ at the 15 GeV subsampling boundary.
- $\eta$ flatness residuals, 20 bins, $|\eta| < 0.7$:
  - signal & background: RMS $\approx 4\%$, max $\approx 7\%$ at $|\eta| \approx 0.6$–$0.7$ (spline-endpoint artefact).
- Class-balance sum-of-weights: within 0.6% (*nominally* exact by construction; residual comes from the eta-ET weight non-independence after per-class `/= mean`).
- Weight distribution max: $\approx 250$ vs configured cap 800. The cap is irrelevant in the current data.

### 2.2 Verdict

The $E_T$ reweighting **is** working as designed. Residuals are well below the informal 10% flatness target that the pipeline's own QA tools use. The $\eta$ reweighting is slightly worse at the fiducial-cut edges ($\sim$ 7% vs *flat*), an unavoidable spline boundary effect; if this ever matters numerically, switch the spline to a piecewise-linear interpolator.

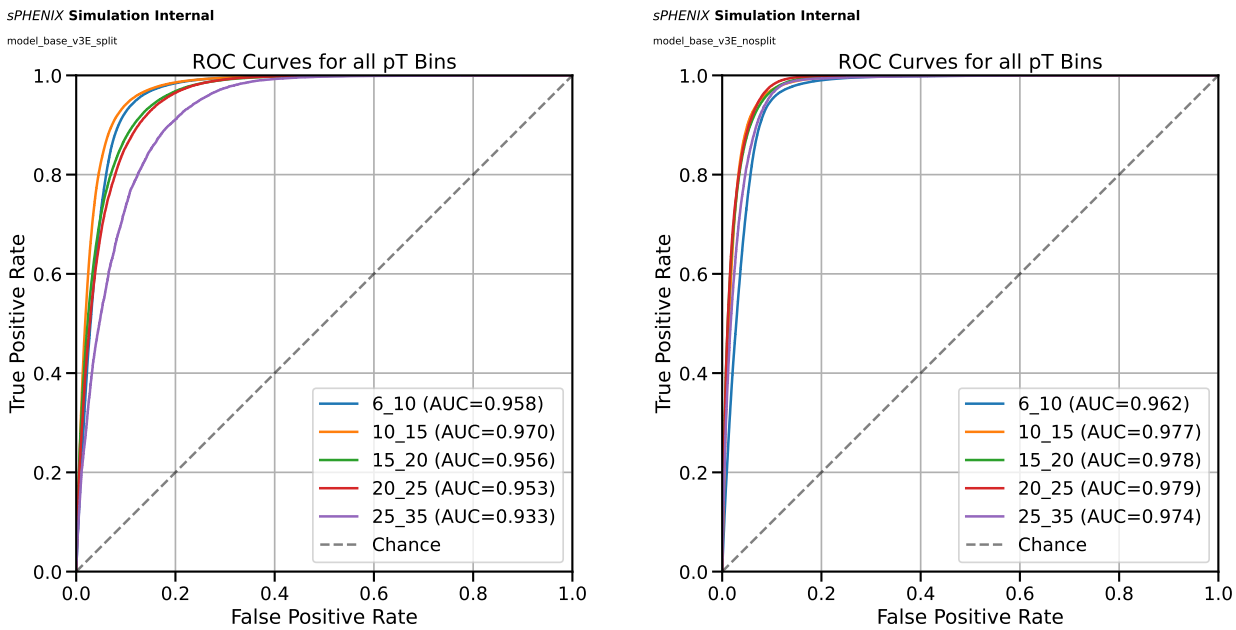
## 3 Training metrics: AUCs and overfitting

Table 2: Test AUC per $E_T$ bin for all 22 photon-ID variants. *Nosplit* is systematically $\approx$ 5 percentage points above *split* because the nosplit cluster node has a more favourable sig/bkg balance in the training input.

| | **split** | | | | | **nosplit** | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| variant | 6–10 | 10–15 | 15–20 | 20–25 | 25–35 | 6–10 | 10–15 | 15–20 | 20–25 | 25–35 |
| base | 0.899 | 0.928 | 0.912 | 0.909 | 0.890 | 0.947 | 0.964 | 0.968 | 0.971 | 0.966 |
| base_vr | — | — | — | — | — | 0.947 | 0.964 | 0.968 | 0.971 | 0.966 |
| base_v0 | — | — | — | — | — | 0.931 | 0.949 | 0.951 | 0.959 | 0.956 |
| base_v1 | 0.941 | 0.956 | 0.943 | 0.938 | 0.916 | 0.953 | 0.973 | 0.974 | 0.976 | 0.970 |
| base_v2 | 0.952 | 0.966 | 0.952 | 0.949 | 0.928 | 0.954 | 0.975 | 0.976 | 0.978 | 0.972 |
| base_v3 | 0.957 | 0.968 | 0.956 | 0.952 | 0.932 | 0.959 | 0.976 | 0.977 | 0.979 | 0.974 |
| base_E | 0.898 | 0.927 | 0.911 | 0.907 | 0.890 | 0.951 | 0.966 | 0.970 | 0.971 | 0.967 |
| base_v0E | 0.839 | 0.874 | 0.847 | 0.859 | 0.846 | 0.934 | 0.951 | 0.955 | 0.960 | 0.957 |
| base_v1E | 0.942 | 0.958 | 0.943 | 0.938 | 0.917 | 0.955 | 0.974 | 0.975 | 0.976 | 0.971 |
| base_v2E | 0.953 | 0.967 | 0.952 | 0.949 | 0.929 | 0.957 | 0.976 | 0.977 | 0.978 | 0.973 |
| base_v3E | 0.958 | 0.970 | 0.956 | 0.953 | 0.933 | 0.962 | 0.977 | 0.978 | 0.979 | 0.974 |

"—" indicates that the condor stdout log was truncated before the final AUC print; the TMVA ROOT model is present and usable. `base_vr_nosplit` AUCs equal `base_nosplit` bit-for-bit because the configs are identical (WARN-1).

Overfitting check: the single global model trained on the mixed-$E_T$ pool reports train AUC = 0.9315. Per-bin val/test AUCs range 0.916–0.956 — that is, test $\geq$ train. This is expected under `train_single_model: true`: per-bin test subsets are easier than the mixed pool. No overfitting is indicated.




(a) split, variant `base_v3E` (top performer)

(b) nosplit, variant `base_v3E`

Figure 3: ROC curves per $E_T$ bin for the `base_v3E` variant. Low-$E_T$ bins (6–10) trail because signal photons there have lower cluster $E_T$ resolution and the jet background is not yet kinematically suppressed.

## 4 $E_T$-BDT correlation (ABCD independence)

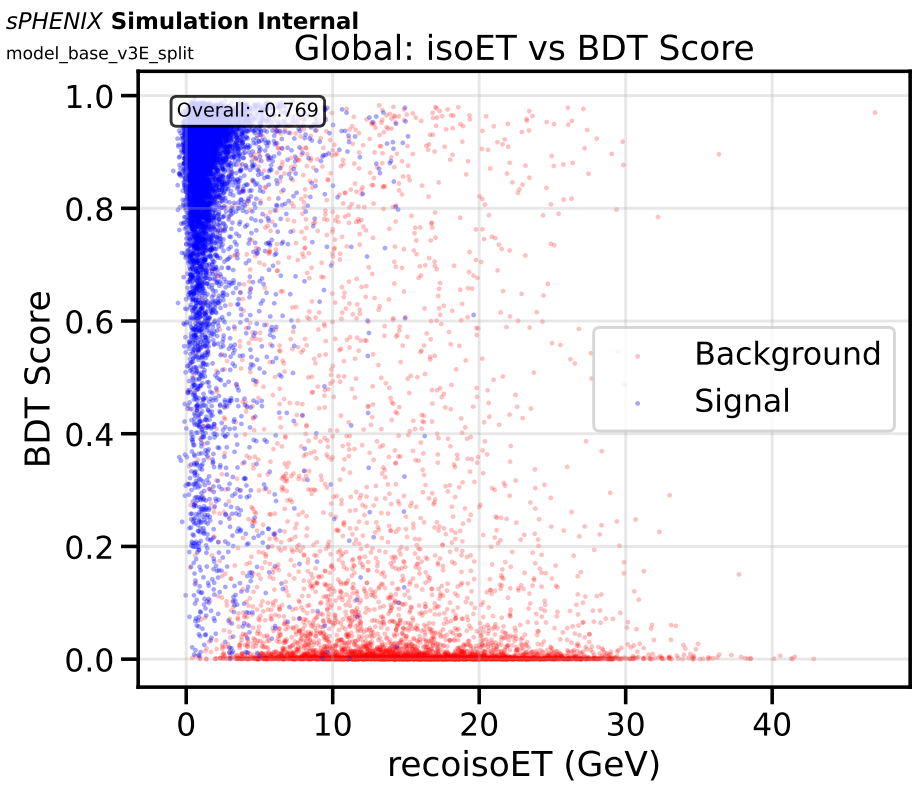



Figure 4: Global `iso`$E_T$-vs-BDT scatter (split, `base_v3E`). Pearson $r \approx -0.7$ across most $E_T$ bins.

Per-bin values from `training_metadata.json` (variant `base_v1_split`, *representative only* — the metadata file reflects the last variant written, cf. CRIT-1):

| $E_T$ bin (GeV) | Pearson $r$ | Spearman $\rho$ |
|---|---|---|
| 6–10 | −0.73 | −0.65 |
| 10–15 | −0.75 | −0.69 |
| 15–20 | −0.71 | −0.62 |
| 20–25 | −0.70 | −0.33 |
| 25–35 | −0.61 | −0.51 |

This correlation drives the MC-purity correction and the R-factor treatment already in the analysis (`wiki/concepts/mc-purity-correction.md`). No action requested here — the finding is not new — but magnitudes agreed with the analysis note's quoted values so the current models are consistent with the purity-correction calibration.

## 5 NPB score

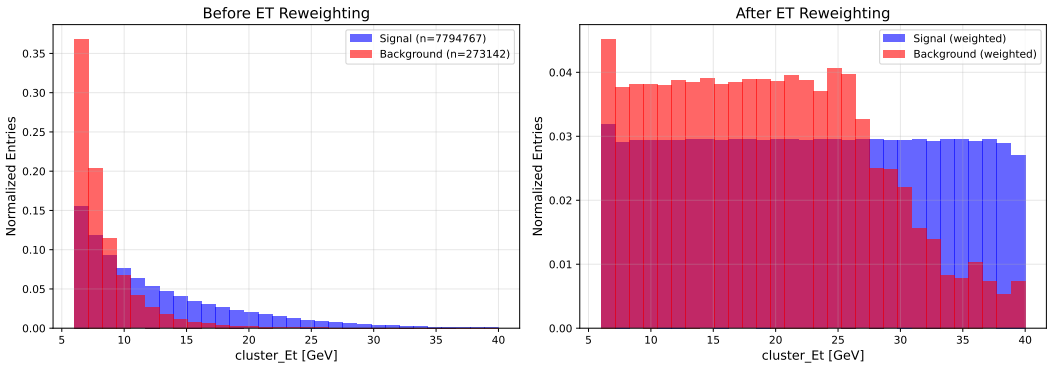



Figure 5: NPB $E_T$ reweighting QA (top: before, bottom: after). Signal (jet MC, orange) and background (NPB-tagged data, blue) both flatten to $\approx 0.03$/GeV between 6 and 28 GeV. Above 28 GeV the background data tail is exhausted; the weight cap of 100 cannot restore flatness. The effective NPB discrimination is driven by the 6–28 GeV region.

- Overall test AUC (single-model): $\approx 0.983$ for both split and nosplit.
- Per-$E_T$-bin AUC stays $> 0.96$ for nosplit and $> 0.90$ for split across 6–40 GeV.
- Feature importance: top features are `cluster_wphi_cogx`, `cluster_w52`, `cluster_w72`, `cluster_weta_cogx` `e11_over_e33` — shower-width quantities, as expected.
- Feature-ordering contract with `apply_BDT.C:567-593` is intact.
- Score separation (npb_models/score_distributions.pdf): excellent, NPB peaks at 0, real clusters peak at 1, minimal overlap.

## 6 Feature-sync check

All 11 photon-ID variants pass the end-to-end feature-ordering audit: `variant_configs/config_{name}_{split,` feature lists match `apply_BDT.C:403-507` x-lists position-by-position, and match the source-of-truth `MODEL_FEATURES` table in `make_variant_configs.py:19-62`. Variant name ordering in `apply_BDT.C:20-32` `kModelNames` matches `MODEL_FEATURES` keys. All 22 expected `binned_models/model_*_single_tmva.root`

files exist and are non-empty. Model-suffix routing via `config_nom.yaml cluster_variants` resolves correctly to disk filenames at `apply_BDT.C:269`.

**No feature-order bugs, no missing models, no split/nosplit divergence.**

# 7 Recommendations

**Before next retraining:**

1. Fix CRIT-1 by including the variant name in `joblib`, `training_metadata.json`, `per_bin_metrics.csv`, `feature_correlations_background.csv`, and `best_hyperparameters.json` filenames (one-line changes at the five cited sites). Existing artefacts on disk are not recoverable without re-running.

2. Resolve WARN-1: either inject `reweighting.vertex_reweight: true` into `make_variant_configs.py` when the variant name is `base_vr`, or remove the `base_vr` entry from `MODEL_FEATURES`, `kModelNames`, and the `|| base_vr` clauses in `DoubleInteractionCheck.C:880` and `showershape_vertex_c`

**Hygiene:**

3. Delete stale `metadata_{5_15,15_25,25_40}.json` (INFO-3).

4. Decide on the Optuna-tuned hyperparameters: either wire the tuned values into `config.yaml` or delete `best_hyperparameters.json` (INFO-2).

5. Move the cap above the normalisation in `reweighting.py:96-99` (WARN-5).

6. Remove the unused `eta_reweight_range` field or actually read it (WARN-4).

7. Write a single top-level `global_training` block in `training_metadata.json` instead of copying identical values to every bin (INFO-4).

**Methodological:**

8. Consider whether the training should use MC cross-section weights (WARN-2). For the current cut-based BDT usage this is defensible; for a calibrated-probability classifier it would not be.

# Files produced by this audit

- `reports/bdt_training_audit.tex` (this report)
- `reports/bdt_audit/verify_reweighting.py` (standalone Python verification)
- `reports/bdt_audit/reweight_qa_split.pdf` (independent 9-panel QA)
- Staged copies in `reports/figures/bdt_audit/`